06. Strings & String Methods

Created by Li Huanyu, last modified by suncaper on Mar 06, 2026

Introduction

In the real world of Data Engineering, data is rarely clean. You will receive files with extra spaces, weird capitalizations, and mixed-up formats. **Strings** (text data) are the most common data type you will clean. In this lesson, we will cover how to slice, chop, and format strings using Python's built-in **String Methods**.

In this lesson, we will cover:

- **Slicing** (Extracting parts of a string).
- **Core Methods:** `split()`, `strip()`, `replace()`, `lower()`/`upper()`.
- **Method Chaining** (Combining multiple operations).

Part 1: String Indexing and Slicing

In Python, a string is just a sequence of characters. Every character has an **index** (a position number). **Crucial Rule:** In Python, counting *always* starts at `0`, not `1`.

1. Indexing (Getting a single character)

```
word = "SPARK"
# S:0, P:1, A:2, R:3, K:4
print(word[0])  # Output: S
print(word[3])  # Output: R
```

2. Slicing (Getting a range of characters) You can extract a substring using the syntax `[start:stop]`. *Note: Python includes the `start` index, but goes UP TO (and excludes) the `stop` index.*

```
date_string = "2024-10-15"

# Extract the Year (indices 0, 1, 2, 3)
year = date_string[0:4]
print(year)  # Output: 2024

# Extract the Month (indices 5, 6)
month = date_string[5:7]
print(month) # Output: 10
```

Part 2: Core String Methods (The Data Cleaning Arsenal)

A "method" is a built-in function that belongs to a specific data type. You call methods using a dot (`.`) after the string variable.

1. `.lower()` and `.upper()`

Standardizes text case. Critical when grouping data (so "Apple" and "apple" are counted as the same thing).

```
raw_city = "nEw YoRk"
clean_city = raw_city.upper()
```

```
print(clean_city)  # Output: NEW YORK
```

2. `.strip()` Removes invisible whitespace (spaces, tabs, newlines) from the *beginning* and *end* of a string.

```
# ❌ Incorrect: Data with hidden spaces will break your database queries
messy_email = "   admin@company.com \n"

# ✅ Correct: Always strip user inputs or log data
clean_email = messy_email.strip()
print(clean_email)  # Output: admin@company.com
```

3. `.replace(old, new)` Swaps specific characters. Extremely useful for cleaning corrupted numbers or standardizing delimiters.

```
# Cleaning a price that came with a comma instead of a decimal
european_price = "1.999,50"
standard_price = european_price.replace(".", "").replace(",", ".")
print(standard_price) # Output: 1999.50
```

4. `.split(separator)` Splits a single string into a **List** of multiple strings based on a separator (like a comma or space).
Big Data Context: This is exactly how a CSV (Comma Separated Values) file is parsed!

```
csv_row = "Alice,Data Engineer,New York"
columns = csv_row.split(",")
print(columns[1])  # Output: Data Engineer
```

Part 3: Python vs. Legacy Languages (Method Chaining)

In data processing, you rarely apply just one rule. You usually need to strip spaces, lower the case, *and* replace characters.

1) Java Example (The "Verbose" Way) In older Java MapReduce jobs, you often had to create intermediate variables:

```
// Java Syntax
String rawData = "   USER_DATA_01  ";
String step1 = rawData.trim();
String step2 = step1.toLowerCase();
String cleanData = step2.replace("_", "-");
```

2) Python Example (The "Fluent" Way)

Python allows **Method Chaining**. Because every string method returns a new string, you can just chain the dots together. It reads left-to-right.

```
# Python Syntax
raw_data = "   USER_DATA_01  "
```

```
clean_data = raw_data.strip().lower().replace("_", "-")
print(clean_data) # Output: user-data-01
```

Part 4: Hands-on Exercise (ETL Simulation)

Let's simulate an **ETL (Extract, Transform, Load)** pipeline. We will extract a messy server log, clean it up, and prepare the individual data points.

1. Open your code editor.
2. Create a file named `string_etl_practice.py`.
3. Type the following code:

```
# Big Data Simulation: Cleaning a Server Log

"""
Scenario: We receive a messy log line from a web server.
We need to extract the Date, Error Level, and User ID.
"""

# 1. EXTRACT: The raw, messy string
raw_log = "   [2024/05/12] -- cRiTiCaL -- UserId:90210   \n"

# 2. TRANSFORM: Clean the text
# - Remove outer spaces/newlines
# - Convert to uppercase for standardization
# - Replace the weird slashes in the date with dashes
clean_log = raw_log.strip().upper().replace("/", "-")
print("Cleaned String:", clean_log)
# Currently looks like: [2024-05-12] -- CRITICAL -- USERID:90210

# 3. TRANSFORM: Split into components
# We see the data is separated by " -- "
log_parts = clean_log.split(" -- ")

# 4. LOAD (Ready for database)
date_part = log_parts[0]     # "[2024-05-12]"
error_level = log_parts[1]   # "CRITICAL"
user_info = log_parts[2]     # "USERID:90210"

# Let's slice the User ID to remove the "USERID:" prefix (which is 7 characters)
final_user_id = user_info[7:]

print("\nFinal Parsed Data:")
print("Date:", date_part)
print("Level:", error_level)
print("User ID:", final_user_id)
```

Run the code. Output:

```
Cleaned String: [2024-05-12] -- CRITICAL -- USERID:90210

Final Parsed Data:
Date: [2024-05-12]
Level: CRITICAL
User ID: 90210
```

Summary

- **Slicing (`[start:stop]`)** lets you extract exact character positions. Remember, Python counts from `0`.
- **`.strip()`** is mandatory for cleaning incoming data. It prevents invisible space bugs.
- **`.split()`** turns one string into many, forming the basis of CSV and log parsing.
- **Method Chaining** (`.strip().lower()`) makes Python code readable, fast, and highly efficient for ETL tasks.

No labels